

Reviewer Pack — Pebble-Cost Gadget Lifts Decision Tree Depth to KW-Game Leng...

Entry 52cebd2ffe62 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 01:38:42 UTC

Pebble-Cost Gadget Lifts Decision Tree Depth to KW-Game Length

Entry ID: 52cebd2ffe62 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 01:38:42 UTC

1. Conjecture

Field A (mathematical branch): Black-white pebbling games on rooted DAGs (Cook-Sethi pebbling cost) **Field B** (complexity object): Karchmer-Wigderson communication complexity on lifted query problems

Statement:

Let f be a partial Boolean function on m bits and let G_f be the canonical AND/OR DAG of its minimum-depth decision tree T_f , with black-white pebbling cost $p(G_f)$. Define the lifted relation $f \circ g$ where g is the 2-bit indexing gadget IND_2 on $b=2$ address bits, and let $KW(f \circ g)$ denote the deterministic KW-game length on $f \circ g$. Then for every f with $|T_f| \leq 2^m$ and $m \leq 12$, $KW(f \circ g) = \text{depth}(T_f) + p(G_f) \pm 1$, i.e. the lifted KW length equals the tree depth plus the pebbling cost of its DAG up to additive 1.

Rationale (proposer's reasoning):

Raz-McKenzie style lifts add a multiplicative gadget overhead, but the *exact* additive correction has never been pinned to a classical DAG invariant; pebbling cost measures exactly the simultaneous-memory needed to simulate the tree as a circuit, which is precisely the resource the KW-prover must spend to track gadget answers. If correct, this gives a tight (not asymptotic) lifting identity computable on tiny instances, sharper than known $O(\text{depth} \cdot \log b)$ bounds. It also explains why IND_2 already suffices for many separations: the gadget cost is absorbed by pebbling rather than by an extra log factor.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 110e5a59e2ba7796

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 200 sampled f (all f for $m \in \{3, 4, 5\}$; ≥ 50 random partial f per $m \in \{6, \dots, 12\}$, each with a fixed seed), compute $d = \text{depth}(T_f)$, $p = p(G_f)$, $k = KW(f \circ IND_2)$. Conjecture is

SUPPORTED iff $|k-d-p| \leq 1$ for 100% of samples; FALSIFIED if any single sample yields $|k-d-p| \geq 2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Karchmer-Wigderson lifting indexing gadget pebbling depth - black-white pebbling decision tree depth communication complexity lifted - KW game lower bound formula depth pebble game DAG gadget composition

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def truth_table_to_function(tt):
        return lambda x: tt[x]

    def min_depth_decision_tree(tt):
        n = len(tt[0])
        m = len(tt)

        @lru_cache(None)
        def dp(i, j):
            if i == n:
                return 1
            if j == m:
                return float('inf')
            if tt[j][i] is None:
                return min(dp(i + 1, j), dp(i, j + 1))
            return 1

        return dp(0, 0)

    def pebbling_cost(G):
```

```

V = len(G)
Q = [(set(), 0)]
visited = set()

while Q:
    (pebbles, cost) = Q.pop(0)
    if frozenset(pebbles) in visited:
        continue
    visited.add(frozenset(pebbles))

    if len(pebbles) == V:
        return cost

    for v in range(V):
        if v not in pebbles and all(u in pebbles for u in G[v]):
            Q.append((pebbles | {v}, cost + 1))

return float('inf')

def kw_game_length(f, g):
    n = len(f)
    m = len(g)
    states = [(set(), set())]
    visited = set()

    while states:
        (X, Y) = states.pop(0)
        if frozenset(X | Y) in visited:
            continue
        visited.add(frozenset(X | Y))

        if len(X) == n and len(Y) == m:
            return len(visited) - 1

        for x in range(n):
            if x not in X:
                states.append((X | {x}, Y))
        for y in range(m):
            if y not in Y:
                states.append((X, Y | {y}))

    return float('inf')

def ind_2_gadget():
    return {
        (0, 0): 0,
        (0, 1): 1,
        (1, 0): 1,
        (1, 1): 0
    }

m = random.randint(3, 5) if m == 6 else m
tt = [[random.choice([True, False]) for _ in range(2**m)] for _ in range(2**m)]
f = truth_table_to_function(tt)
T_f = min_depth_decision_tree(f)
G_f = build_and_or_dag(f)

```

```

p_G_f = pebbling_cost(G_f)

g = ind_2_gadget()
f_comp_g = lambda x: f(g[x[0], x[1]])
k = kw_game_length(f_comp_g, g)

return {
    "metric_name": "KW-Game Length",
    "metric_value": k,
    "instances_tested": 1,
    "conjecture_holds": abs(k - T_f - p_G_f) <= 1,
    "counterexample": "" if abs(k - T_f - p_G_f) <= 1 else f"Depth: {T_f}, Pebbling Cost: {p_G_f}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(abs(r["metric_value"] - r["instances_tested"]) >= 2 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if abs(result["metric_value"] - result["instances_tested"]) >= 2)
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}'\n first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db2bed9c.py", line 116, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_db2bed9c.py", line 92, in run_trial
    m = random.randint(3, 5) if m == 6 else m
        ^

```

UnboundLocalError: cannot access local variable 'm' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an UnboundLocalError before producing any data, so the pre-registered support condition (≥ 200 samples with $|k-d-p| \leq 1$ for 100%) was never evaluated. No counterexample was observed either, so neither support nor falsification is established. | next: Fix the variable-scoping bug in `run_trial` (initialize `m` before the conditional reassignment) and rerun the full sweep over $m \in \{3, \dots, 12\}$ to obtain the required ≥ 200 samples.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	21840
2	preregistration	claude_max	opus	0	0	5820
3	novelty	claude_max	opus	0	0	3944
4	novelty	claude_max	opus	0	0	7175
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15288
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11787
7	judge	claude_max	opus	0	0	5312

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 71166 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/52cebd2ffe62.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/52cebd2ffe62.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/52cebd2ffe62.tar.gz` (if generated)